

Laboratory work 1. ERD diagram.

1. Description

Airport: Stores airport information: airport_id, airport_name, country, state, city, created_at and updated_at.

Airline: Stores airline information: airline_id, airline_code, name, country, created_at and updated_at.

Flight: Stores scheduled and actual flight information, including airline_id, departure and arrival airport IDs, gates, scheduled times, actual times, created_at and updated_at.

Booking: Represents a passenger booking for a flight. It contains booking_id, flight_id, passenger_id, status, booking_platform, ticket_price, created_at and updated_at.

Booking Change: Stores changes made to a booking in a separate table: booking_change_id, booking_id, changed_field, old_value, new_value, created_at and updated_at.

Passenger: Stores passenger profile information: passenger_id, first name, last name, gender, date of birth, country of citizenship, country of residence, passport number, created_at and updated_at.

Boarding Pass: Stores boarding pass information: boarding_pass_id, booking_id, seat, boarding_time, created_at and updated_at.

Baggage: Stores baggage registered for a booking: baggage_id, booking_id, weight_kg, created_at and updated_at.

Baggage Check: Stores baggage-check information: baggage_checking_id, booking_id, passenger_id, check_results, created_at and updated_at.

Security Check: Stores passenger security-check information: security_check_id, passenger_id, check_results, created_at and updated_at.

2. Relationships and Cardinality

Relationship	Cardinality	Explanation
Airline → Flight	1:N	One airline can operate many flights; each flight references exactly one airline.
Airport → Flight (departure)	1:N	One airport can be the departure airport of many flights; each flight references exactly one departure airport.
Airport → Flight (arrival)	1:N	One airport can be the arrival airport of many flights; each flight references exactly one arrival airport.

Relationship	Cardinality	Explanation
Flight → Booking	1:N	One flight can have many bookings; each booking references exactly one flight.
Passenger → Booking	1:N	One passenger can have many bookings; each booking references exactly one passenger.
Booking → Booking Change	1:N	One booking can have many recorded changes; each change references exactly one booking.
Booking → Boarding Pass	1:1	A booking receives at most one boarding pass. Implemented by a UNIQUE constraint on booking_id in the boarding pass table.
Booking → Baggage	1:N	One booking can have many pieces of baggage; each piece references exactly one booking.
Booking → Baggage Check	1:N	One booking can have many baggage-check records; each record references exactly one booking.
Passenger → Baggage Check	1:N	One passenger can have many baggage-check records; each record references exactly one passenger.
Passenger → Security Check	1:N	One passenger can have many security-check records; each record references exactly one passenger.

PK — primary key; FK — foreign key; UNIQUE — an attribute or a combination of attributes whose value may not repeat. 1:N means one-to-many and 1:1 means one-to-one. The diagram uses the FK structure to show which child record references a parent record.

Participation constraints follow from the NOT NULL declarations. Every foreign key in the schema is declared NOT NULL, therefore the participation of the child entity is always mandatory: a flight cannot exist without an airline and two airports, a booking cannot exist without a flight and a passenger, and baggage, boarding passes, baggage checks and booking changes cannot exist without their booking. The participation of the parent entity is optional in every relationship: an airport may have no flights, a flight may have no bookings, a booking may have no baggage and no boarding pass, and a passenger may have no security check record yet.

Unique attributes used in the schema are airline_code, passport_number, the pair (airport_name, city), the pair (flight_id, passenger_id) in Booking, and booking_id in Boarding Pass. The last one is what turns the Booking to Boarding Pass relationship into one-to-one.

There is no direct M:N relationship shown explicitly in the schema. The logical many-to-many relationship between flights and passengers is resolved by the Booking entity, which holds both foreign keys together with its own attributes, while the UNIQUE constraint on (flight_id, passenger_id) prevents the same passenger from being booked twice on the same flight.



